

An Algebraic Foundation for Knowledge Graph Construction

Anonymous Authors

Abstract. Although they exist since more than ten years already, have attracted diverse implementations, and have been used successfully in a significant number of applications, declarative mapping languages for constructing knowledge graphs from heterogeneous types of data sources still lack a solid formal foundation. This makes it impossible to introduce implementation and optimization techniques that are provably correct and, in fact, has led to discrepancies between different implementations. Moreover, it precludes studying fundamental properties of different languages (e.g., expressive power). To address this gap, this paper introduces a language-agnostic algebra for capturing mapping definitions. As further contributions, we show that the popular mapping language RML can be translated into our algebra (by which we also provide a formal definition of the semantics of RML) and we prove several algebraic rewriting rules that can be used to optimize mapping plans based on our algebra.

1 Introduction

Knowledge graphs (KGs) play a vital role in enabling semantic interoperability, integration, and querying. They allow systems to execute advanced semantic queries, facilitating the integration and reasoning of the data at hand. The importance of KGs is further amplified by the growing adoption of large language models (LLMs) alongside KGs for tasks like retrieval-augmented generation [22]. However, many existing data infrastructures lack the capability to generate KGs dynamically, and migrating legacy databases with diverse data formats to KG-based systems can be costly for organizations. Unlocking the wealth of knowledge in these legacy systems require the generation of KGs from heterogeneous data efficiently. One such approach of generating KGs involves the usage of declarative mapping languages [21].

Application of declarative mapping languages has gained traction in recent years across diverse domains such as COVID research [15], railway networks [14], and incident management in ICT systems [20]. A plethora of mapping languages exist to support these practical applications. On one hand, query-based languages like SPARQL-Generate [10] aim to extend SPARQL for KG generation from heterogeneous data. On the other hand, dedicated mapping languages, such as the RDF Mapping Language (RML) [5], use custom syntax. Furthermore, a single mapping language can have multiple implementations for generating KGs. For example, RML has several implementations [7,8,1,18,16], each offering distinct advantages for their own unique use cases (i.e., FlexRML [7] for KG generation on IoT devices).

State-of-the-art mapping engines exhibit varying operational semantics due to the lack of well-defined formal foundation in the mapping languages they are built upon. This absence of formalizations complicates verifying the correctness of the mapping engines and studying the expressiveness of the mapping languages. Additionally, it poses challenges in optimizing the mappings described by the mapping languages.

In this paper, we fill the gap in the lack of formalizations in mapping languages by providing mapping language-agnostic formalizations with three contributions. The first contribution is the introduction of formal definitions and the proposal of a set of algebraic mapping operators. These operators can be composed to formally define the mapping of heterogeneous data to KG in a language-agnostic manner (Sections 3 and 4). A language-agnostic formalization of mapping heterogeneous data to KG lays the foundation for a unified mapping engine that can work with the user’s preferred mapping language. This approach ensures verified correctness and enables optimizations across various mapping languages. The second contribution demonstrates the expressiveness of our formalisms by applying them to a specific mapping language. Specifically, we present an algorithm that translates RML documents into expressions composed of the algebraic operators defined in our formalism (Section 5). This algorithm shows that our formalism can represent every mapping definable in RML, proving it to be at least as expressive as RML while also providing formal semantics for RML. Lastly, as our final contribution, we introduce a few algebraic equivalences to show the potential of algebraic expression rewriting and thus, enabling the optimizations of mapping heterogeneous data to KG (Section 6).

2 Related Work

We categorize mapping languages into two categories following the work of Van Assche et al. [21] as follows: i) *dedicated languages* which are built specifically for the purpose of KG construction, such as RML [5] and R2RML [19], or use custom syntaxes, and ii) *repurposed languages* which repurpose existing non-mapping languages, such as query languages, to support KG construction. Do note that we adapted the original definition for the category on *dedicated languages* to also include R2RML which does not extend any mapping languages. In this section, we briefly review existing works on formalizations of declarative mapping languages for KG construction.

Formalizations in Repurposed Mapping Languages. To the best of our knowledge, only SPARQL-based mapping languages have defined formal semantics [10,11,3,2]. SPARQL-Anything [2] utilizes a two-step approach. First, the authors formalize the Façade-X meta-model [4], which is used to transform heterogeneous data into an intermediate RDF graph describing the structure of data with their values. Lastly, they use SPARQL CONSTRUCT query on the Façade-X RDF graph for generating the KG, ensuring full compatibility with the formal semantics of SPARQL. SPARQL-Generate [10] extends SPARQL 1.1 to

support mapping heterogeneous data to KG, thus, inheriting the formal semantics of SPARQL. XSPARQL [3] combines the semantics of XQuery and SPARQL for mapping between XML and RDF data. The authors extend their work to support mapping relational databases to RDF data by utilizing the semantics of relational algebra [11]. The aforementioned works rely on the semantics of the underlying non-mapping languages. However, we argue that such reliance complicates the application of these formalisms to dedicated mapping languages.

Formalizations in Dedicated Mapping Languages. R2RML [19] is a dedicated mapping language for R2RML mappings, which are RDF graphs for mapping relational databases to RDF KG. R2RML has served as the basis for several mapping languages that extend its scope from relational databases to structured or semi-structured data [21], such as RML [5]. Thus, it is important to consider the formalization studies on R2RML, as they can impact languages within the R2RML family.

Sequeda et al. [17] provide formalization to the semantics of R2RML using Datalog. The authors prove that the semantics of R2RML can be captured using 57 distinct Datalog rules to generate RDF from relational databases. While this formalization is focused on relational databases, our approach is designed to capture arbitrary types of heterogeneous data sources for mapping. Furthermore, Datalog-based formalization adds complexity when implementing compliant mapping engines due to the lack of control-flow description. In contrast, our approach clearly defines algebraic operators and introduces control-flow, making it easier to adopt and implement a functional mapping engine. Besides R2RML, a few studies on formalization of RML have been published in recent years [12,8]. Iglesias et al. [8] provide formal semantics to the process of KG generation through Horn clauses. However, their primary focus is optimizing the KG creation process for efficiency, with formalizations as a by-product to support their optimization techniques. It would also be interesting to explore applying their formalization to mapping languages beyond RML. An initial study provided formal semantics for the mappings in a language-agnostic manner through *algebraic mapping operators* [12]. This preliminary work does not cover all aspects of the mappings that can be described using a dedicated mapping language like RML (i.e., joining multiple data sources). Another limitation of this paper is that the definitions are only *semi-formal*; with some presented informally or relying on undefined concepts. Thus, there is still a lack of study in providing language-agnostic, complete, and well-defined semantics for mapping heterogeneous data to KG.

3 Data Model

This section introduces the data model based on which our mapping algebra is defined. Hence, this model captures the types of intermediate results of any mapping process described by our formalism. Informally, the data model is a version of the relational data model, restricted to a special kind of relations, that we call *mapping relations* and that contain RDF terms as possible values.

Table 1. Mapping relation r discussed in Examples 1 and 2.

	a_x	a_s	a_p	a_o	a_g
t	("12", xsd:integer)	ex:alice	foaf:knows	ex:bob	rr:defaultGraph
t'	ex:alice	ex:alice	("knows", xsd:string)	ex:charles	ex:g1
t''	ϵ	ex:bob	foaf:name	("Bob", xsd:string)	ex:g2

As a preliminary step for defining the model formally, we introduce relevant concepts of RDF: We let $\mathcal{S}$ be the countably infinite set of strings (sequences of Unicode code points) and $\mathcal{I}$ be the subset of $\mathcal{S}$ that consists of all strings that are valid IRIs. Moreover, $\mathcal{L}$ is a countably infinite set of pairs $(lex, dt) \in \mathcal{S} \times \mathcal{I}$, which we call *literals*. For each such literal $\ell = (lex, dt)$, lex is its *lexical form* and dt is its *datatype IRI*.¹ Furthermore, we assume a countably infinite set $\mathcal{B}$ of *blank nodes*, which is disjoint from both $\mathcal{S}$ and $\mathcal{L}$. The set $\mathcal{T}$ of all *RDF terms* is defined as $\mathcal{T} = \mathcal{I} \cup \mathcal{B} \cup \mathcal{L}$, and we write $\bar{\mathcal{T}}$ to denote the set of all possible sequences of terms; i.e., each element of $\bar{\mathcal{T}}$ is a sequence of elements of $\mathcal{T}$. As usual, an *RDF triple* is a tuple $(s, p, o) \in (\mathcal{I} \cup \mathcal{B}) \times \mathcal{I} \times \mathcal{T}$, an *RDF graph* is a set of RDF triples, and an *RDF dataset* is a set $\{G_{\text{dft}}, (n_1, G_1), \dots, (n_m, G_m)\}$ where $G_{\text{dft}}, G_1, \dots, G_m$ are RDF graphs, $m \geq 0$, and $n_i \in \mathcal{I} \cup \mathcal{B}$ for all $i \in \{1, \dots, m\}$.

As additional ingredients for defining mapping relations, we assume a countably infinite set $\mathcal{A}$ of *attributes* and a special value ϵ that is not an RDF term (i.e., $\epsilon \notin \mathcal{T}$) and that shall be used to capture processing errors. The last ingredient are *mapping tuples*, which are the type of tuples used in mapping relations.

Definition 1. A **mapping tuple** is a partial function $t : \mathcal{A} \rightarrow \mathcal{T} \cup \{\epsilon\}$.

We say that two mapping tuples t and t' are *compatible* if, for every attribute $a \in \text{dom}(t) \cap \text{dom}(t')$, it holds that $t(a) = t'(a)$. Given two mapping tuples t and t' that are compatible, we write $t \cup t'$ to denote the mapping tuple t'' that is the merge of t and t' ; that is, $\text{dom}(t'') = \text{dom}(t) \cup \text{dom}(t')$ and, for every attribute $a \in \text{dom}(t'')$, $t''(a) = t(a)$ if $a \in \text{dom}(t)$, and $t''(a) = t'(a)$ otherwise.

Now we are ready to define our notion of a mapping relation.

Definition 2. A **mapping relation** r is a tuple (A, I) , where $A \subset \mathcal{A}$ is a finite, non-empty set of attributes and I is a set of mapping tuples such that, for every such tuple $t \in I$, it holds that $\text{dom}(t) = A$. We call A the **schema** of the mapping relation and I is the **instance** of the mapping relation.

For the sake of conciseness, we sometimes write $t \in r$ to denote that t is a mapping tuple in the instance I of a mapping relation $r = (A, I)$.

Example 1. Consider the mapping relation $r = (A, I)$ that is represented in a tabular form in Table 1. It holds that $A = \{a_x, a_s, a_p, a_o, a_g\}$ and I contains three mapping tuples, t , t' , and t'' , such that $t(a_x)$ is the literal ("12", xsd:integer), both $t'(a_x)$ is the IRI ex:alice, $t''(a_x) = \epsilon$, etc.

¹ To avoid overly complex formulas in this paper, we ignore the option of literals that have a language tag. Yet, the formalism can easily be extended to cover them.

While mapping relations are intermediate results of mapping processes described by our algebra, the ultimate goal of such processes is to produce an RDF dataset. Therefore, we also need to define how mapping relations are mapped into RDF datasets. To this end, we assume four special attributes: $a_s, a_p, a_o, a_g \in \mathcal{A}$. Every mapping relation whose schema contains these attributes is mapped to an RDF dataset by using the values that the tuples in the relation have for these attributes. That is, if the values that a mapping tuple has for a_s , a_p , and a_o form a valid RDF triple, then the graph determined by the a_g -value of the tuple contains this triple. Formally, we define this approach as follows.

Definition 3. Let $r = (A, I)$ be a mapping relation with $a_s, a_p, a_o, a_g \in A$, and

$$I_{\text{valid}} = \{t \in I \mid (t(a_s), t(a_p), t(a_o)) \text{ is an RDF triple and } t(a_g) \in \mathcal{I} \cup \mathcal{B}\} \text{ and} \\ N = \{t(a_g) \in \mathcal{I} \cup \mathcal{B} \mid t \in I_{\text{valid}} \text{ and } t(a_g) \text{ is not the IRI } \texttt{rr:defaultGraph}\}.$$

The **RDF dataset resulting from r** is the RDF dataset

$$D = \{G_{\text{dft}}\} \cup \{(n, G_n) \mid n \in N\},$$

where, for every $n \in N$,

$$G_n = \{(t(a_s), t(a_p), t(a_o)) \mid t \in I_{\text{valid}} \text{ such that } t(a_g) = n\} \text{ and} \\ G_{\text{dft}} = \{(t(a_s), t(a_p), t(a_o)) \mid t \in I_{\text{valid}} \text{ such that } t(a_g) \text{ is } \texttt{rr:defaultGraph}\}.$$

Example 2. For the mapping relation in Example 1 it holds that $I_{\text{valid}} = \{t, t''\}$ and $N = \{\texttt{ex:g2}\}$. Therefore, the RDF dataset resulting from this mapping relation is $D = \{G, (\texttt{ex:g2}, G')\}$ with $G = \{(\texttt{ex:alice}, \texttt{foaf:knows}, \texttt{ex:bob})\}$ and $G' = \{(\texttt{ex:bob}, \texttt{foaf:name}, (\texttt{"Bob"}, \texttt{xsd:string}))\}$. Notice that $t' \notin I_{\text{valid}}$ because $(t'(a_s), t'(a_p), t'(a_o))$ is not an RDF triple (since $t'(a_p)$ is a literal).

4 Mapping Algebra

This section introduces our mapping algebra which consists of five types of operators. The output of each such operator is a mapping relation and the inputs (if any) are mapping relations as well. Hence, the operators can be combined into arbitrarily complex algebra expressions, where each such expression defines a mapping from one or more sources of (semi-)structured data into an RDF dataset.²

4.1 Source Operator

The basis of every expression that defines a mapping using our algebra are operators that establish a relational view of each source of input data. From the perspective of the algebra, these *source operators* are nullary operators; they

² We assume here that the mapping relation resulting from the root operator of the expression contains the aforementioned attributes a_s , a_p , a_o , and a_g (see Definition 3).

do not have any mapping relation as input. The mapping relation that such an operator creates as output depends on the data of the corresponding data source and can be specified based on a language appropriate to identify and extract relevant pieces of data from the data source. As our mapping algebra should be applicable for various types of data sources, for which different such languages are suitable, we introduce an abstraction of this extraction process and define our notion of source operators based on this abstraction.

Our abstraction considers two infinite sets, $\mathcal{D}$ and $\mathcal{Q}$, which are disjoint. $\mathcal{D}$ is the set of all possible *data objects*. In practice, there are different kinds of data objects, even within the context of the same type of data source, and data objects may be nested within one another. For instance, each line of a CSV file may be seen as a data object, and so may each value within such a line, as well as the CSV file as a whole. In our abstraction, any particular kind of data objects is considered as a specific subset of $\mathcal{D}$, and to indicate that a data object $d \in \mathcal{D}$ is contained in another data object $D \in \mathcal{D}$, we write $d \in D$.

The set $\mathcal{Q}$ captures possible *query languages* to retrieve data objects from other data objects. We consider each such language $L \in \mathcal{Q}$ as a set, where every element $q \in L$ is one of the expressions admitted by the language. In practice, each such language is designed to be used for a specific kind of data objects (i.e., a specific subset of $\mathcal{D}$). Therefore, we define the notion of the *types of data sources* considered for the source operators of our algebra as a structure that specifies the relevant kinds of data objects and the corresponding evaluation semantics of the languages to be used for data sources of such a type.

Definition 4. A **source type** is a tuple $(\mathcal{D}^{\text{ds}}, \mathcal{D}^{\text{c1}}, \mathcal{D}^{\text{c2}}, L, L', eval, eval', cast)$ where

- $\mathcal{D}^{\text{ds}} \subseteq \mathcal{D}$, $\mathcal{D}^{\text{c1}} \subseteq \mathcal{D}$, $\mathcal{D}^{\text{c2}} \subseteq \mathcal{D}$,
- L and L' are query languages in $\mathcal{Q}$,
- $eval$ is a function $eval: \mathcal{D}^{\text{ds}} \times L \rightarrow \overline{\mathcal{D}^{\text{c1}}}$,
- $eval'$ is a partial function $eval': \mathcal{D}^{\text{ds}} \times \mathcal{D}^{\text{c1}} \times L' \rightarrow \overline{\mathcal{D}^{\text{c2}}}$, and
- $cast$ is a partial function $cast: \mathcal{D}^{\text{c2}} \rightarrow \mathcal{L}$,

where $\overline{\mathcal{D}^{\text{c1}}}$, resp. $\overline{\mathcal{D}^{\text{c2}}}$, is the set of all sequences of data objects in $\mathcal{D}^{\text{c1}}$, resp. $\mathcal{D}^{\text{c2}}$.

Definition 5. A **data source** s is a tuple $(type, D)$ in which $type$ is a source type $(\mathcal{D}^{\text{ds}}, \mathcal{D}^{\text{c1}}, \mathcal{D}^{\text{c2}}, L, L', eval, eval', cast)$ and $D \in \mathcal{D}^{\text{ds}}$.

As per Definitions 4 and 5, the set $\mathcal{D}^{\text{ds}}$ of a source type determines the kind of data objects that the data sources of this type may provide access to. For instance, for CSV-based data sources this may be the (infinite) set of all possible CSV files. The functions $eval$ and $eval'$ are meant to define the evaluation semantics of the languages L and L' , respectively. Notice that the domain of $eval'$ consists of 3-tuples that contain an additional data object as their second element, which is meant as a context object for the evaluation. The idea of the source operators shall be to use L to specify relevant context objects and, then, to use L' to retrieve values for creating a mapping tuple per context object.

While a formal definition of concrete source types is beyond the scope of this paper, the following two examples outline informally how JSON-based data sources and CSV-based data sources may be captured within our abstraction.

Example 3. To define $type_{\text{json}} = (\mathcal{D}_{\text{json}}^{\text{ds}}, \mathcal{D}_{\text{jp}}^{\text{c1}}, \mathcal{D}_{\text{jp}}^{\text{c2}}, L_{\text{jp}}, L'_{\text{jp}}, eval_{\text{jp}}, eval'_{\text{jp}}, cast_{\text{json}})$ as a source type for JSON-based data sources, we let $\mathcal{D}_{\text{json}}^{\text{ds}}$ be the infinite set of all possible JSON documents, both $\mathcal{D}_{\text{jp}}^{\text{c1}}$ and $\mathcal{D}_{\text{jp}}^{\text{c2}}$ are the infinite set of all possible elements that may occur within JSON documents (i.e., $\mathcal{D}_{\text{jp}}^{\text{c1}} = \mathcal{D}_{\text{jp}}^{\text{c2}}$), L_{jp} is the set of all JSONPath expressions that begin with the selector for the root element (i.e., $\$$), and L'_{jp} are all JSONPath expressions that do not begin with $\$$. Then, $eval_{\text{jp}}$ and $eval'_{\text{jp}}$ capture the evaluation semantics of JSONPath. That is, for a JSON document $D \in \mathcal{D}_{\text{json}}^{\text{ds}}$ and a query $q \in L_{\text{jp}}$, $eval_{\text{jp}}(D, q)$ returns the elements of D selected by q . If $q' \in L'_{\text{jp}}$, and assume $d \in \mathcal{D}_{\text{jp}}^{\text{c1}}$ is a JSON object within D , then $eval'_{\text{jp}}(D, d, q')$ returns the elements of D that can be reached from d as per the traversal specified by q' . Finally, $cast_{\text{json}}$ maps JSON values to RDF literals. For instance, string values may be mapped to `xsd:string` literals, numeric values that are integers may be mapped to `xsd:integer` literals, etc.

Example 4. As a source type for CSV-based data sources, we may introduce $type_{\text{csv}} = (\mathcal{D}_{\text{csv}}^{\text{ds}}, \mathcal{D}_{\text{csv}}^{\text{c1}}, \mathcal{D}_{\text{csv}}^{\text{c2}}, L_{\text{csv}}, L'_{\text{csv}}, eval_{\text{csv}}, eval'_{\text{csv}}, cast_{\text{csv}})$ as follows. $\mathcal{D}_{\text{csv}}^{\text{ds}}$ is the infinite set of all possible CSV files, $\mathcal{D}_{\text{jp}}^{\text{c1}}$ is the infinite set of all possible rows of CSV files, $\mathcal{D}_{\text{jp}}^{\text{c2}} \subset \mathcal{S}$ is the infinite set of strings that may be values within CSV files, and the $cast_{\text{csv}}$ function maps all these string values in $\mathcal{D}_{\text{jp}}^{\text{c2}}$ to `xsd:string` literals. As the language L_{csv} , we may use the singleton set that contains (only) the empty string ε , and $L'_{\text{csv}} \subset \mathcal{S}$ is the set of all strings that may be used as column names in CSV files. Informally, $eval_{\text{csv}}$ and $eval'_{\text{csv}}$ may then be defined as follows. For every CSV file $D \in \mathcal{D}_{\text{csv}}^{\text{ds}}$, $eval_{\text{csv}}(D, \varepsilon)$ returns all rows of D , and for every such row $d \in eval_{\text{csv}}(D, \varepsilon)$ and every string c that is a column name in D , $eval'_{\text{csv}}(D, d, c)$ is the value that the row d has in column c .

Example 5. As the basis of a running example for the rest of this section, we assume a data source $s_{\text{ex}} = (type_{\text{csv}}, D_{\text{ex}})$ where D_{ex} is a CSV file with four columns, named `id`, `firstname`, `lastname`, and `age`, and the following two rows:

$$(1, \text{Alice}, \text{Lee}, 23) \quad \text{and} \quad (2, \text{Bob}, \text{Malice}, \text{unknown}).$$

If we write d_1 to denote the first of these rows, then $eval'_{\text{csv}}(D_{\text{ex}}, d_1, \text{firstname})$ is the string `Alice`, and $cast_{\text{csv}}(\text{Alice})$ is the RDF literal `("Alice", xsd:string)`.

Now we are ready to define our notion of source operators. While they are nullary operators, they are specified using three parameters: the data source from which the data for the produced mapping relation is extracted, a query that selects relevant context objects from the data source, and a map that associates attributes with queries, where these queries are meant to select the values that a mapping tuple produced for a context object has for the attributes. Formally, we define the mapping relation produced by a source operator as follows.

Definition 6. Let $s = (type, D)$ be a data source with $type = (\mathcal{D}^{\text{ds}}, \mathcal{D}^{\text{c1}}, \mathcal{D}^{\text{c2}}, L, L', eval, eval', cast)$, $q \in L$, and $\mathbb{P}$ be a partial function $\mathbb{P}: \mathcal{A} \rightarrow L'$. The $(q, \mathbb{P})$ -**specific mapping relation obtained from s** , denoted by $\text{Source}^{(s, q, \mathbb{P})}$, is the

mapping relation (A, I) such that $A = \text{dom}(\mathbb{P})$ and

$$I = \left\{ \{a_1 \rightarrow \text{cast}(v_1), \dots, a_n \rightarrow \text{cast}(v_n)\} \mid d \in \bar{O} \text{ and } ((a_1, v_1), \dots, (a_n, v_n)) \in X_d \right\},$$

where $\bar{O} = \text{eval}(D, q)$ and

$$X_d = \bigtimes_{a \in \text{dom}(\mathbb{P})} \{(a, v) \mid v \in \text{eval}'(D, d, q') \text{ with } q' = \mathbb{P}(a)\}.$$

Example 6. The result of $\text{Source}^{(s_{\text{ex}}, \epsilon, \mathbb{P}_{\text{ex}})}$ with source s_{ex} of Example 5 and $\mathbb{P}_{\text{ex}} = \{a_1 \rightarrow \text{id}, a_2 \rightarrow \text{firstname}, a_3 \rightarrow \text{age}\}$ is the relation $(\{a_1, a_2, a_3\}, \{t_1, t_2\})$ with

$$\begin{aligned} t_1 = \{a_1 \rightarrow ("1", \text{xsd:string}), & \quad \text{and} \quad t_2 = \{a_1 \rightarrow ("2", \text{xsd:string}), \\ a_2 \rightarrow ("Alice", \text{xsd:string}), & \quad a_2 \rightarrow ("Bob", \text{xsd:string}), \\ a_3 \rightarrow ("23", \text{xsd:string})\} & \quad a_3 \rightarrow ("unknown", \text{xsd:string})\}. \end{aligned}$$

4.2 Extend Operator

Extend operators are unary operators that can be used to extend a mapping relation with an additional attribute. The values that the tuples of the relation shall have for this attribute are specified via a so-called *extend expression* that can be formed using *extension functions*. In the following, we first introduce the notions of such functions and such expressions, and then define extend operators.

Definition 7. An **extension function** is a function f of the form

$$f: (\mathcal{T} \cup \{\epsilon\}) \times \dots \times (\mathcal{T} \cup \{\epsilon\}) \rightarrow (\mathcal{T} \cup \{\epsilon\}).$$

Example 7. A concrete example of such an extension function may be a unary function called `toInt` that converts `xsd:string` literals representing integers into `xsd:integer` literals. Formally, for every $v \in (\mathcal{T} \cup \epsilon)$, we define:

$$\text{toInt}(v) = \begin{cases} (lex, \text{xsd:integer}) & \text{if } v \text{ is a literal } (lex, dt) \text{ such that } lex \text{ is in the} \\ & \text{lexical space of the datatype denoted by the} \\ & \text{IRI } \text{xsd:integer} \text{ and } dt \text{ is the IRI } \text{xsd:string}, \\ \epsilon & \text{else.} \end{cases}$$

Definition 8. **Extend expressions** are defined recursively as follows:

1. Every RDF term in $\mathcal{T}$ is an extend expression.
2. Every attribute in $\mathcal{A}$ is an extend expression.
3. If $\varphi_1, \dots, \varphi_n$ are extend expressions and f is an extension function (as per Definition 7), then the tuple $(f, \varphi_1, \dots, \varphi_n)$ is an extend expression.

For every extend expression φ , we write $\text{attrs}(\varphi)$ to denote the set of all attributes mentioned in φ . Formally, this set is defined recursively as follows.

1. If φ is an RDF term, then $\text{attrs}(\varphi) = \emptyset$.
2. If φ is an attribute $a \in \mathcal{A}$, then $\text{attrs}(\varphi) = \{a\}$.
3. If φ is of the form $(f, \varphi_1, \dots, \varphi_n)$, then $\text{attrs}(\varphi) = \bigcup_{i \in \{1, \dots, n\}} \text{attrs}(\varphi_i)$.

While Definition 8 introduces the syntax of extend expressions, their interpretation as a function to obtain values for mappings tuples is defined as follows.

Definition 9. Let φ be an extend expression and t be a mapping tuple. The **evaluation of φ over t** , denoted by $\text{eval}(\varphi, t)$, is either an RDF term or the error value (ϵ), and is determined recursively as follows:

$$\text{eval}(\varphi, t) = \begin{cases} \varphi & \text{if } \varphi \text{ is an RDF term,} \\ t(\varphi) & \text{if } \varphi \in \mathcal{A} \text{ and } \varphi \in \text{dom}(t), \\ f(\text{eval}(\varphi_1, t), \dots, \text{eval}(\varphi_n, t)) & \text{if } \varphi \text{ is of the form } (f, \varphi_1, \dots, \varphi_n) \\ & \text{such that } f \text{ is an } n\text{-ary function,} \\ \epsilon & \text{else.} \end{cases}$$

Example 8. If φ_{ex} is the extend expression (toInt, a_3) , for the tuples t_1 and t_2 of Example 6, $\text{eval}(\varphi_{\text{ex}}, t_1)$ is the literal $(\text{"23"}, \text{xsd:integer})$ and $\text{eval}(\varphi_{\text{ex}}, t_2) = \epsilon$.

Now we can define the semantics of extend operators as follows.

Definition 10. Let $r = (A, I)$ be a mapping relation, a be an attribute that is not in A (i.e., $a \in \mathcal{A} \setminus A$), and φ be an extend expression. The **(a, φ) -extension of r** , denoted by $\text{Extend}_{\varphi}^a(r)$, is the mapping relation (A', I') such that $A' = A \cup \{a\}$ and $I' = \{t \cup \{a \rightarrow \text{eval}(\varphi, t)\} \mid t \in I\}$.

Example 9. Let r be the mapping relation of Example 6. We may extend this relation with an attribute a_4 for which each of the two tuples of r gets the value obtained by applying the extend expression φ_{ex} of Example 8. This operation can be captured as $\text{Extend}_{\varphi_{\text{ex}}}^{a_4}(r)$ and results in the mapping relation $(\{a_1, a_2, a_3, a_4\}, \{t'_1, t'_2\})$ with

$$t'_1 = t_1 \cup \{a_4 \rightarrow (\text{"23"}, \text{xsd:integer})\} \quad \text{and} \quad t'_2 = t_2 \cup \{a_4 \rightarrow \epsilon\}.$$

While the example shows how extend operators can be used to convert literals from one datatype to another, other use cases including adding the same constant RDF term to all tuples of a mapping relation, creating IRIs from other attribute values, and applying arbitrary functions to one or more other attribute values.

4.3 Relational Algebra Operators

In addition to the operators introduced above, any operator of the relational algebra can also be used for mapping relations. For instance, the following definition introduces the notion of *projection*, adapted to mapping relations.

Definition 11. Let $r = (A, I)$ be a mapping relation and $P \subseteq A$ be a non-empty subset of the attributes of r . The **P -specific projection of r** , denoted by $\text{Project}^P(r)$, is the mapping relation (P, I') such that $I' = \{t[P] \mid t \in I\}$, where, for every mapping tuple $t \in I$, $t[P]$ denotes the mapping tuple t' that is the restriction of t to P ; i.e., $\text{dom}(t') = P$ and $t'(a) = t(a)$ for all $a \in P$.

Example 10. Let r is the mapping relation of Example 9. Assuming that further steps of a potential mapping process that involve this relation use only the attributes a_2 and a_4 of r , we may project away the rest of the attributes. To this end, we use $\text{Project}^{\{a_2, a_4\}}(r)$ and obtain the relation $(\{a_2, a_4\}, \{t'_1, t'_2\})$ with

$$\begin{aligned} t'_1 &= \{a_2 \rightarrow (\text{"Alice"}, \text{xsd:string}), a_4 \rightarrow (\text{"23"}, \text{xsd:integer})\} \text{ and} \\ t'_2 &= \{a_2 \rightarrow (\text{"Bob"}, \text{xsd:string}), a_4 \rightarrow \epsilon\}. \end{aligned}$$

As two more examples of adopting traditional relational algebra operators for mapping relations, we present the following two definitions.

Definition 12. Let $r_1 = (A_1, I_1)$ and $r_2 = (A_2, I_2)$ be mapping relations such that $A_1 \cap A_2 = \emptyset$, and $\mathbb{J} \subseteq A_1 \times A_2$. The **$\mathbb{J}$ -based equijoin of r_1 and r_2** , denoted by $\text{EqJoin}^{\mathbb{J}}(r_1, r_2)$, is the mapping relation (A, I) such that $A = A_1 \cup A_2$ and

$$I = \{t_1 \cup t_2 \mid t_1 \in I_1 \text{ and } t_2 \in I_2 \text{ such that } t_1(a_1) = t_2(a_2) \text{ for all } (a_1, a_2) \in \mathbb{J}\}.$$

Definition 13. Let $r_1 = (A_1, I_1)$ and $r_2 = (A_2, I_2)$ be mapping relations such that $A_1 = A_2$. The **union of r_1 and r_2** , denoted by $\text{Union}(r_1, r_2)$, is the mapping relation $(A_1, I_1 \cup I_2)$.

5 Algebra-Based Definition of RML

This section introduces an algorithm to convert mappings defined using the mapping language RML (specifically, version 1.1.2 [6]) into our mapping algebra. This algorithm shows that *our algebra is at least as expressive as RML*. Moreover, through this algorithm, in combination with the algebra, we provide a *formal definition of the semantics of RML mappings*. Hereafter, we describe the algorithm step by step, while a complete pseudocode representation is given as Algorithm 1. For the description, we assume familiarity with the concepts of RML [5,6,9].

Input and Output. Since RML mappings are captured as RDF graphs that use the RML ontology [9], the main input to our algorithm is such an RDF graph. As a second input, we assume an injective function $S2B: \mathcal{S} \rightarrow \mathcal{B}$ that maps every string to a unique blank node (which shall become relevant later). The output is a mapping relation that is obtained by applying operators of our algebra and that can be converted into an RDF dataset as per Definition 3.

Normalization Phase. To minimize the variations of RML descriptions to be considered, the algorithm begins by converting the given RML mapping into a *normal form* in which i) no shortcut properties are used, ii) all referencing object

Algorithm 1: Translates an RML mapping into our mapping algebra.

Input: G - an RDF graph (assumed to describe an RML mapping)
 $S2B$ - an injective function that maps every string to a unique blank node

Output: a mapping relation with attributes a_s , a_p , a_o , and a_g

```

1  $G_{NF} \leftarrow$  normalized version of  $G$  (apply the update queries of Appendix ?? to  $G$ )
2  $\omega_{spog} \leftarrow$  the empty mapping relation  $(\{a_s, a_p, a_o, a_g\}, \emptyset)$ 
3 foreach  $u_{tm} \in \mathcal{I} \cup \mathcal{B}$  for which  $(u_{tm}, \text{rdf:type}, \text{rr:TriplesMap}) \in G_{NF}$  do
4    $\omega \leftarrow \text{Source}^{(s,q,\mathbb{P})}$ , where  $(s,q) = \text{SrcAndRootQuery}(u_{tm}, G_{NF})$  // Definition 14
      and  $\mathbb{P} = \text{ExtractQueries}(u_{tm}, G_{NF})$  // Algorithm 2
5    $u_{sm} \leftarrow o$ , where  $o \in \mathcal{I} \cup \mathcal{B}$  such that  $(u_{tm}, \text{rr:subjectMap}, o) \in G_{NF}$ 
6    $u_{pom} \leftarrow o$ , where  $o \in \mathcal{I} \cup \mathcal{B}$  such that  $(u_{tm}, \text{rr:predicateObjectMap}, o) \in G_{NF}$ 
7    $u_{pm} \leftarrow o$ , where  $o \in \mathcal{I} \cup \mathcal{B}$  such that  $(u_{pom}, \text{rr:predicateMap}, o) \in G_{NF}$ 
8    $u_{om} \leftarrow o$ , where  $o \in \mathcal{I} \cup \mathcal{B}$  such that  $(u_{pom}, \text{rr:objectMap}, o) \in G_{NF}$ 
9    $\omega \leftarrow \text{Extend}_{\varphi}^{a_s}(\omega)$ , where  $\varphi = \text{CreateExtExpr}(u_{sm}, G_{NF}, S2B, \mathbb{P})$  // Algorithm 3
10   $\omega \leftarrow \text{Extend}_{\varphi}^{a_p}(\omega)$ , where  $\varphi = \text{CreateExtExpr}(u_{pm}, G_{NF}, S2B, \mathbb{P})$ 
11  if there is a  $u_{ptm} \in \mathcal{I} \cup \mathcal{B}$  such that  $(u_{om}, \text{rr:parentTriplesMap}, u_{ptm}) \in G_{NF}$  then
12     $\mathbb{J} \leftarrow \emptyset$ 
13    foreach  $u_{jc} \in \mathcal{I} \cup \mathcal{B}$  for which  $(u_{om}, \text{rr:joinCondition}, u_{jc}) \in G_{NF}$  do
14       $a \leftarrow \mathbb{P}^{-1}(lex)$ , where  $(u_{jc}, \text{rr:child}, o) \in G_{NF}$  with  $o = (lex, dt) \in \mathcal{L}$ 
15       $a' \leftarrow \mathbb{P}^{-1}(lex)$ , where  $(u_{jc}, \text{rr:parent}, o) \in G_{NF}$  with  $o = (lex, dt) \in \mathcal{L}$ 
16       $\mathbb{J} \leftarrow \mathbb{J} \cup \{(a, a')\}$ 
17     $\omega \leftarrow \text{EqJoin}^{\mathbb{J}}(\omega, \text{Source}^{(s',q',\mathbb{P}')}(\omega))$ , where  $\mathbb{P}' = \text{ExtractQueries}(u_{ptm}, G_{NF})$ 
      and  $(s',q') = \text{SrcAndRootQuery}(u_{ptm}, G_{NF})$ 
18     $u_{sptm} \leftarrow o$ , where  $o \in \mathcal{I} \cup \mathcal{B}$  such that  $(u_{ptm}, \text{rr:subjectMap}, o) \in G_{NF}$ 
19     $\omega \leftarrow \text{Extend}_{\varphi}^{a_o}(\omega)$ , where  $\varphi = \text{CreateExtExpr}(u_{sptm}, G_{NF}, S2B, \mathbb{P}')$ 
20  else
21     $\omega \leftarrow \text{Extend}_{\varphi}^{a_o}(\omega)$ , where  $\varphi = \text{CreateExtExpr}(u_{om}, G_{NF}, S2B, \mathbb{P})$ 
22  if there is a  $u_{gm} \in \mathcal{I} \cup \mathcal{B}$  such that  $(u_{sm}, \text{rr:graphMap}, u_{gm}) \in G_{NF}$  or
       $(u_{pom}, \text{rr:graphMap}, u_{gm}) \in G_{NF}$  then
23     $\omega \leftarrow \text{Extend}_{\varphi}^{a_g}(\omega)$ , where  $\varphi = \text{CreateExtExpr}(u_{gm}, G_{NF}, S2B, \mathbb{P})$ 
24  else
25     $\omega \leftarrow \text{Extend}_{\varphi}^{a_g}(\omega)$ , where  $\varphi$  is the IRI  $\text{rr:defaultGraph}$ 
26   $\omega_{spog} \leftarrow \text{Union}(\omega_{spog}, \text{Project}^{\{a_s, a_p, a_o, a_g\}}(\omega))$ 
27 return  $\omega_{spog}$ 

```

maps have a join condition, and iii) every triples map has only a single predicate-object map with a single predicate map, a single object map, and at most one graph map. Every RML mapping can be converted into this normal form by applying the following sequence of rewriting steps, which we have adopted from Rodríguez-Muro and Rezk [13], with adaptations specific to RML. We present these steps with the accompanying update queries which captures the steps formally. None of these steps changes the meaning of the defined mapping.

1. Every class IRI definition is expanded into a predicate-object map. (Listing 1.1)

Listing 1.1. Normalization step 1 (expand shortcuts for class IRIs).

```

DELETE { ?sm rr:class ?sm_class . }
INSERT { ?tm rr:predicateObjectMap [
  rr:predicateMap [
    rr:constant rdf:type ;
    rr:termType rr:IRI ; ] ;
  rr:objectMap [
    rr:termType rr:IRI ;
    rr:constant ?sm_class ;
  ] ; ] . }
WHERE { ?tm rr:subjectMap ?sm .
        ?sm rr:class ?sm_class . }

```

Listing 1.2. Normalization step 3 (duplicate multi-predicate-object maps into singletons).

```

DELETE{ ?tm rr:predicateObjectMap ?pom.
        ?pom rr:predicateMap ?pm;
        rr:objectMap ?om; rr:graphMap ?gm. }
INSERT{ ?tm rr:predicateObjectMap [
  rr:predicateMap ?pm;
  rr:objectMap ?om;
  rr:graphMap ?gm; ]. }
WHERE{ ?tm rr:predicateObjectMap ?pom.
        ?pom rr:predicateMap ?pm.
        ?pom rr:objectMap ?om.
        OPTIONAL { ?pom rr:graphMap ?gm. } }

```

Listing 1.3. Normalization step 2 (expand shortcuts for constant-valued term maps).

```

DELETE { ?tm rr:subject ?sm_constant . ?pompm rr:predicate ?pm_constant .
        ?termMap rr:graph ?gm_constant . ?pomom rr:object ?om_constant . }
INSERT { ?tm rr:subjectMap [ rr:constant ?sm_constant; ].
        ?pompm rr:predicateMap [ rr:constant ?pm_constant; ].
        ?pomom rr:objectMap [ rr:constant ?om_constant; ].
        ?termMap rr:graphMap [ rr:constant ?gm_constant; ]. }
WHERE { { ?tm rr:subject ?sm_constant }
        UNION { ?pompm rr:predicate ?pm_constant }
        UNION { ?pomom rr:object ?om_constant }
        UNION { ?termMap rr:graph ?gm_constant } }

```

Listing 1.4. Normalization step 4 (replace referencing object maps that do not have a join condition).

```

DELETE{ ?om rr:parentTriplesMap ?ptm . }
INSERT{ ?om rr:reference ?ref ;
        rr:template ?template ;
        rr:constant ?const ;
        rr:termType rr:IRI . }
WHERE{ ?om rr:parentTriplesMap ?ptm .
        ?ptm rr:subjectMap ?sm .
        OPTIONAL{ ?sm rr:reference ?ref . }
        OPTIONAL{ ?sm rr:template ?template . }
        OPTIONAL{ ?sm rr:constant ?const . }
        FILTER NOT EXISTS
        { ?om rr:joinCondition ?jc . } }

```

Listing 1.5. Normalization step 5 (duplicate triples maps that contain multiple predicate-object maps).

```

DELETE{ ?tm a rr:TriplesMap;
        rml:logicalSource ?ls; rr:subjectMap ?sm;
        rr:predicateObjectMap ?pom . }
INSERT{ [ ] a rr:TriplesMap;
        rml:logicalSource ?ls;
        rr:subjectMap ?sm;
        rr:predicateObjectMap ?pom . }
WHERE{ ?tm rml:logicalSource ?ls;
        rr:subjectMap ?sm;
        rr:predicateObjectMap ?pom . }

```

2. All shortcut properties for constant-valued term maps are expanded. (Listing 1.3)
3. Predicate-object maps with multiple predicate maps or multiple object maps are duplicated to have a single predicate map and a single object map each. (Listing 1.2)
4. Referencing object maps without a join condition are replaced by an object map with the subject IRI of the parent triples map as the reference value. (Listing 1.4)
5. Every triples map with multiple predicate-object maps is duplicated such that each of the resulting triples maps has a single predicate-object map only. (Listing 1.5)
6. Every triples map with multiple graph maps is duplicated such that each of the resulting triples maps has one of these graph maps. (Listing 1.6)

Main Loop. After normalization, the algorithm iterates over all triples maps defined by the (normalized) RML mapping (lines 3–26). For each of them, it combines algebra operators, as described below, to define a mapping relation that, in the end, has the attributes a_s , a_p , a_o , and a_g . These mapping relations

Listing 1.6. Normalization step 6 (duplicate triples maps to contain one graph map).

```

DELETE { ?tm rml:logicalSource ?ls ; rr:subjectMap ?sm ; rr:predicateObjectMap ?pom ;
        a rr:TriplesMap .
        ?sm rr:graphMap ?gm .
        ?pom rr:graphMap ?gm2 . }
INSERT { [] rml:logicalSource ?ls ; rr:subjectMap ?sm ; rr:predicateObjectMap ?pom ;
        a rr:TriplesMap .
        ?sm rr:graphMap ?gm .
        [] rml:logicalSource ?ls ; rr:subjectMap ?sm ; rr:predicateObjectMap ?pom ;
        a rr:TriplesMap .
        ?sm rr:graphMap ?gm2 . }
WHERE { ?tm rml:logicalSource ?ls ; rr:subjectMap ?sm ; rr:predicateObjectMap ?pom .
        ?sm rr:graphMap ?gm .
        ?pom rr:graphMap ?gm2 . }

```

Listing 7. RML mapping with one triples map (written in Turtle, prefix decl. omitted).

```

ex:tm rml:logicalSource [ rml:source "data.csv";
                        rml:referenceFormulation ql:CSV ];
  rr:subjectMap [ rr:template "http://example.com/person_{ID}";
                rr:termType rr:IRI ];
  rr:predicateObjectMap [ rr:predicate rdfs:label;
                        rr:objectMap [rml:reference "Name"] ].

```

are combined using union (line 26), resulting in a mapping relation (line 27) that captures the complete RDF dataset defined by the given RML mapping.

Translation of the Logical Source. As the first step for each triples map, the algorithm creates a source operator (line 4). The first two parameters for this operator—the data source s and the query expression q for selecting relevant context objects (cf. Definition 6)—depend on the logical source defined for the triples map. Since RML is extensible regarding the types of logical sources, we have to abstract the creation of these two parameters by the following function.

Definition 14. Given an IRI or blank node u_{tm} (assumed to denote an RML triples map) and an RDF graph G (assumed to describe an RML mapping, including u_{tm}), $\text{SRCANDROOTQUERY}(u_{tm}, G)$ is a pair (s, q) of a data source $s = (type, D)$ with $type = (\mathcal{D}^{ds}, \mathcal{D}^{c1}, \mathcal{D}^{c2}, L, L', eval, eval', cast)$ and a query expression $q \in L$, as extracted from the description of the logical source of u_{tm} in G .

Example 11. Given an RDF graph G_{ex} that contains the RML mapping of Listing 7, $\text{SRCANDROOTQUERY}(ex:tm, G_{ex}) = (s'_{ex}, \varepsilon)$ where $s'_{ex} = (type_{csv}, D'_{ex})$ such that $type_{csv}$ is the source type for CSV-based data sources (cf. Example 4), D'_{ex} is the CSV file mentioned in the second triple, and ε is the empty string.

The third parameter for the source operator—the partial function $\mathbb{P}$ that maps attributes to query expressions (cf. Definition 6)—is populated by Algorithm 2, which extracts the query expressions from the term maps of the given triples map. In particular, the term maps may use query expressions as references (lines 3–4 of Algorithm 2), in string templates (lines 5–6), and as child and parent queries of join conditions (lines 7–10). To focus only on the term maps of the given triples map, the extraction is restricted to the relevant subgraph of the given RDF graph (line 1 of Algorithm 2), which we define formally as follows.

Algorithm 2: EXTRACTQUERIES - extract query expressions from triples maps.

Input: u_{tm} - IRI or blank node of the triples map to extract queries from
 G - an RDF graph (assumed to contain a *normalized* RML description)
Output: an injective partial function $\mathbb{P}$ that maps attributes to query expressions

```

1  $G_{tm} \leftarrow$  the  $u_{tm}$ -rooted subgraph of  $G$  // Definition 15
2  $Q \leftarrow \emptyset$  // Initially empty set of query expressions
3 forall  $s \in \mathcal{I} \cup \mathcal{B}$  and  $o \in \mathcal{L}$  for which there is a triple  $(s, rml:reference, o) \in G_{tm}$  do
4    $Q \leftarrow Q \cup \{lex\}$ , where  $o = (lex, dt)$ 
5 forall  $s \in \mathcal{I} \cup \mathcal{B}$  and  $o \in \mathcal{L}$  for which there is a triple  $(s, rr:template, o) \in G_{tm}$  do
6    $Q \leftarrow Q \cup Q'$ , where  $o = (lex, dt)$  and  $Q'$  is the set of all substrings of  $lex$  that
   are enclosed by "{" and "}"
7  $T_c \leftarrow \{(s, p, o) \in G_{tm} \mid p \text{ is the IRI } rr:child \text{ and } o \in \mathcal{L}\}$ 
8  $T_p \leftarrow \{(s, p, o) \in G \mid p \text{ is the IRI } rr:parent, o \in \mathcal{L}, \text{ and there is an } s' \in \mathcal{I} \cup \mathcal{B} \text{ s.t.}$ 
    $(s', rr:joinCondition, s) \in G \text{ and } (s', rr:parentTriplesMap, u_{tm}) \in G\}$ 
9 foreach triple  $(s, p, o) \in (T_c \cup T_p)$  do
10    $Q \leftarrow Q \cup \{lex\}$ , where  $o = (lex, dt)$ 
11  $\mathbb{P} \leftarrow$  (initially) empty function from attributes to query expressions
12 foreach query expression  $q \in Q$  do
13    $\mathbb{P} \leftarrow \mathbb{P} \cup \{a \rightarrow q\}$ , where  $a$  is a fresh attribute from  $\mathcal{A} \setminus \{a_s, a_p, a_o, a_g\}$  that
   has not been used before in the whole translation process
14 return  $\mathbb{P}$ 

```

Definition 15. Let G be an RDF graph and $u \in \mathcal{I} \cup \mathcal{B}$. The u -rooted subgraph of G is an RDF graph $G' \subseteq G$ that is defined recursively as follows:

1. G' contains every triple $(s, p, o) \in G$ for which it holds that $s = u$.
2. G' contains every triple $(s, p, o) \in G$ for which there is already another triple $(s', p', o') \in G'$ such that $s = o'$ and p' is not the IRI `rr:parentTriplesMap`.

Example 12. For the RDF graph G_{ex} that contains the RML mapping of Listing 7, EXTRACTQUERIES(`ex:tm`, G_{ex}) returns $\mathbb{P} = \{a_1 \rightarrow ID, a_2 \rightarrow Name\}$, where a_1 and a_2 are arbitrary attributes such that $a_1 \neq a_2$ and $a_1, a_2 \notin \{a_s, a_p, a_o, a_g\}$.

Translation of the Subject Map and the Predicate Map. The source operator constructed in the previous step creates a mapping relation with values obtained via the extracted query expressions (along the lines of Example 6). This relation can now be extended by using these values to create RDF terms as defined by the term maps of the triples map. To this end, for the subject (term) map and the predicate (term) map, the algorithm adds an extend operator, respectively (lines 9–10 in Algorithm 1). The construction of the extend expressions of these operators (see Definitions 8 and 10) is captured in Algorithm 3 and uses the following extension functions.

Algorithm 3: CREATEEXTEXPR - creates an extend expression for a term map.

Input: u - IRI or blank node of the term map to create the extend expression for
 G - an RDF graph (assumed to describe the term map u)
 $S2B$ - an injective function that maps every string to a unique blank node
 $\mathbb{P}$ - an injective partial function that maps attributes to query expressions

Output: an extend expression (as per Definition 8)

```

1 if  $G$  contains a triple  $(u, rr:constant, o)$  then return  $o$ 
2  $\varphi \leftarrow$  initially empty extend expression
3 if  $G$  contains a triple  $(u, rr:reference, o)$  such that  $o$  is a literal  $(lex, dt)$  then
4    $\varphi \leftarrow a$ , where  $a$  is the attribute in  $\text{dom}(\mathbb{P})$  such that  $\mathbb{P}(a) = lex$ 
5 else if  $G$  contains a triple  $(u, rr:template, o)$  such that  $o$  is a literal  $(lex, dt)$  then
6    $\bar{S} \leftarrow \text{SPLIT}(lex)$  // Definition 16
7    $\varphi \leftarrow (s_1, xsd:string)$ , where  $s_1$  is the first element in  $\bar{S}$ 
8   foreach  $s_i$  in  $\bar{S}$  without the first element do // Iterate based on the order in  $\bar{S}$ 
9     if  $s_i$  is a string that starts with "{" and ends with "}" then
10       $\varphi \leftarrow \text{concat}(\varphi, \mathbb{P}^{-1}(q))$ , where  $q$  is  $s_i$  without enclosing "{" and "}"
11    else
12       $\varphi \leftarrow \text{concat}(\varphi, (s_i, xsd:string))$ 
13 if  $G$  contains the triple  $(u, rr:termType, rr:BlankNode)$  then return  $\text{toBNode}^{S2B}(\varphi)$ 
14 else if  $G$  contains the triple  $(u, rr:termType, rr:IRI)$  then return  $\text{toIRI}(\varphi)$ 
15 else if  $G$  contains a triple  $(u, rr:datatype, o)$  s.t.  $o \in \mathcal{I}$  then return  $\text{toLiteral}(\varphi, o)$ 
16 else return  $\text{toLiteral}(\varphi, xsd:string)$ 

```

For every $v \in (\mathcal{T} \cup \epsilon)$, $dt \in (\mathcal{T} \cup \epsilon)$, and $v' \in (\mathcal{T} \cup \epsilon)$, we define:

$$\text{toIRI}(v) = \begin{cases} lex & \text{if } v \text{ is a literal } (lex, dt) \text{ such that } lex \text{ is a valid IRI} \\ & \text{and } dt \text{ is the IRI } xsd:string, \\ \epsilon & \text{else.} \end{cases}$$

$$\text{toBNode}^{S2B}(v) = \begin{cases} S2B(lex) & \text{if } v \text{ is a literal } (lex, dt) \text{ s.t. } dt = xsd:string, \\ \epsilon & \text{else.} \end{cases}$$

$$\text{toLiteral}(v, dt) = \begin{cases} (lex, dt) & \text{if } v \text{ is a literal } (lex, dt') \text{ s.t. } dt = xsd:string, \\ \epsilon & \text{else.} \end{cases}$$

$$\text{concat}(v, v') = \begin{cases} (lex \circ lex', xsd:string) & \text{if } v \text{ and } v' \text{ are literals } (lex, dt) \text{ and } \\ & (lex', dt'), \text{ respectively, such that} \\ & dt = dt' = xsd:string, \\ \epsilon & \text{else,} \end{cases}$$

where $lex \circ lex'$ is the string obtained from concatenating the strings lex and lex' .

The extend expression created by Algorithm 3 depends on whether the given term map is constant-valued (line 1), reference-valued (lines 3–4), or template-valued (lines 5–12). In the latter case, the extend expression needs to be created such that it instantiates the corresponding template string. To this end, this string is first split into a sequence of substrings (line 6), defined as follows.

Definition 16. For every string $tmpl \in \mathcal{S}$, we write $SPLIT(tmpl)$ to denote a sequence $\bar{S}$ of strings that is constructed by the following two steps.

1. Partition $tmpl$ into a sequence $\bar{S}$ of *query substrings* and *normal substrings* in the order in which these substrings appear in $tmpl$ from left to right, where
 - a *query substring* is a substring starting with an opening curly brace (i.e., "{") and ending with a closing curly brace (i.e., "}"), and
 - a *normal substring* is every substring between two query substrings, as well as the one before the first and the one after the last query substring.
2. If $\bar{S}$ is the empty sequence, insert the empty string as a new substring into $\bar{S}$.

The extend expression is then created to concatenate the resulting substrings (lines 7–12), where every query substring " $\{q\}$ " is replaced by the attribute a for which it holds that $\mathbb{P}(a) = q$, because that attribute holds the values that the source operator obtains for the query expression q .

Example 13. Assume b_{sm} and b_{om} are the blank nodes denoting the subject map and the object map in RDF graph G_{ex} in Listing 7, respectively, and $\mathbb{P} = \{a_1 \rightarrow ID, a_2 \rightarrow Name\}$ as in Example 12. Then, $CREATEEXTEXPR(b_{sm}, G_{ex}, S2B, \mathbb{P})$ returns $toIRI(concat(\ell, a_1))$ where ℓ is the literal ("`http://example.com/person_`", `xsd:string`); $CREATEEXTEXPR(b_{om}, G_{ex}, S2B, \mathbb{P})$ is $toLiteral(a_2, \text{xsd:string})$.

Translation of the Object Map. For object (term) maps, the algorithm also has to consider the case of referencing object maps (lines 11–19 in Algorithm 1), which capture a join with RDF terms defined by the subject map of another triples map. In this case, another source operator needs to be added (for this other triples map) and the mapping relation created by that source operator needs be joined with the mapping relation produced before (line 17). To this end, an equijoin operator is used, for which the join attributes are determined based on the join condition defined for the referencing object map (lines 12–16). Ordinary object maps are handled like subject and predicate maps before (line 21).

Translation of the Graph Map. Finally, if the triples map has a graph map, the algorithm adds an extend operator, with attribute a_g , to create IRIs defined by this graph map (lines 22–23). If the triples map has no graph map, an extend operator that assigns a_g to the constant IRI `rr:defaultGraph` is added (line 25).

6 Algebraic Equivalences

In addition to providing a basis to formally define the semantics of a mapping language such as RML, our mapping algebra may also be used as a foundation

for implementing such languages in mapping tools. More precisely, such tools may use our algebra as a form of language to internally represent the plans that they create for converting data according to a given mapping description, and then they may use algebraic equivalences to rewrite initial mapping plans into more efficient ones (with the provable guarantee that the rewritten plans produce the same result!). While an extensive study of such optimization opportunities is beyond the scope of this paper, in this section we show a number of such equivalences and describe how they may be relevant for a plan optimizer.

6.1 Projection Pushing

The first group of equivalences focuses on pushing project operators deeper into a given plan. This may be useful to reduce the size of intermediate mapping relations in terms of values for attributes that are not used anymore later in the plan. Dropping such attributes from intermediate mapping relations as early as possible reduces the memory space required for executing the mapping plan.

As a typical example of an algebraic equivalence that an optimizer may use for this purpose, consider the following result, which shows that a project operator over an extend-based subexpression may be pushed into this subexpression if the set of projection attributes contains both the extend attribute and the attributes mentioned by the corresponding extend expression (if any).

Proposition 1. Let $r = (A, I)$ be a mapping relation, $a \in \mathcal{A}$ be an attribute that is not in A (i.e., $a \notin A$), φ be an extend expression, and $P \subseteq A$ be a non-empty subset of the attributes of r . If $\text{attrs}(\varphi) \cap A \subseteq P$, then it holds that

$$\text{Project}^{P \cup \{a\}}(\text{Extend}_{\varphi}^a(r)) = \text{Extend}_{\varphi}^a(\text{Project}^P(r)).$$

Proof. We assume that $\text{attrs}(\varphi) \cap A \subseteq P$, and let $(A_1, I_1) = \text{Project}^{P \cup \{a\}}(\text{Extend}_{\varphi}^a(r))$ and $(A_2, I_2) = \text{Extend}_{\varphi}^a(\text{Project}^P(r))$. To show that $(A_1, I_1) = (A_2, I_2)$ we first note that, by Definition 11, it holds that $A_1 = P \cup \{a\}$, and by Definition 10, $A_2 = P \cup \{a\}$. Hence, $A_1 = A_2$ and, thus, it remains to show that $I_1 = I_2$.

We first show that $I_1 \subseteq I_2$, for which we consider an arbitrary mapping tuple $t \in I_1$ and show that $t \in I_2$. Given that $t \in I_1$, by Definition 11, it holds that i) $\text{dom}(t) = P \cup \{a\}$ and ii) there exists a mapping tuple $t' \in \text{Extend}_{\varphi}^a(r)$ such that $t(a') = t'(a')$ for all $a' \in P \cup \{a\}$. Since $t' \in \text{Extend}_{\varphi}^a(r)$, by Definition 10, it holds that i) $\text{dom}(t') = A \cup \{a\}$ and ii) there exists a mapping tuple $t'' \in r$ such that $\text{eval}(\varphi, t'') = t'(a)$ and $t''(a') = t'(a')$ for all $a' \in A$. Now, let t''' be the restriction of t'' to P ; i.e., $t''' = t''[P]$, which, by Definition 11, also means that $\text{dom}(t''') = P$ and $t''' \in \text{Project}^P(r)$. Then, since $\text{attrs}(\varphi) \cap A \subseteq P$ and $t'''(a') = t''(a')$ for all $a' \in P$, it holds that $\text{eval}(\varphi, t''') = \text{eval}(\varphi, t'')$. As a consequence, there exists a tuple $t^{(4)} \in \text{Extend}_{\varphi}^a(\text{Project}^P(r))$ such that i) $\text{dom}(t^{(4)}) = \text{dom}(t''') \cup \{a\}$, ii) $t^{(4)}(a) = t'(a)$, and iii) $t^{(4)}(a') = t'''(a')$ for all $a' \in P$. Since $\text{dom}(t''') \cup \{a\} = P \cup \{a\} = \text{dom}(t)$ and $t'(a) = t(a)$ and $t'''(a') = t''(a') = t'(a') = t(a')$ for all $a' \in P$, we can conclude that $t^{(4)} = t$ and, thus, $t \in \text{Extend}_{\varphi}^a(\text{Project}^P(r))$; i.e., $t \in I_2$.

Now we show that $I_1 \supseteq I_2$, for which we consider a tuple $t \in I_2$ and show that $t \in I_1$. Given that $t \in I_2$, by Definitions 10 and 11, it holds that i) $\text{dom}(t) = \{a\} \cup P$ and ii) there exists a mapping tuple $t' \in \text{Project}^P(r)$ such that $t(a) = \text{eval}(\varphi, t')$ and $t(a') = t'(a')$ for all $a' \in P$. Since $t' \in \text{Project}^P(r)$, it holds that i) $\text{dom}(t') = P$ and ii) there exists a mapping tuple $t'' \in r$ such that $\text{dom}(t'') = A$ and $t''(a') = t'(a')$ for all $a' \in P$. Now, let t''' be the mapping tuple for which it holds that i) $\text{dom}(t''') = A \cup \{a\}$, ii) $t'''(a) = \text{eval}(\varphi, t'')$, and iii) $t'''(a') = t''(a')$ for all $a' \in A$. Hence, by Definition 10, $t''' \in \text{Extend}_\varphi^a(r)$. Next, let $t^{(4)}$ be the restriction of t''' to $P \cup \{a\}$; i.e., $t^{(4)} = t'''[P \cup \{a\}]$, which, by Definition 11, means that $t^{(4)} \in \text{Project}^{P \cup \{a\}}(\text{Extend}_\varphi^a(r))$. Putting everything together, we have that i) $\text{dom}(t^{(4)}) = P \cup \{a\} = \text{dom}(t)$, ii) $t^{(4)}(a) = t'''(a) = \text{eval}(\varphi, t'') = \text{eval}(\varphi, t') = t(a)$, and iii) $t^{(4)}(a') = t'''(a') = t''(a') = t(a)$ for all $a' \in P$. As a consequence, $t^{(4)} = t$ and, thus, $t \in \text{Project}^{P \cup \{a\}}(\text{Extend}_\varphi^a(r))$; i.e., $t \in I_1$. $\square$

To satisfy the condition for the equivalence in Proposition 1 and, thus, to facilitate projection pushing based on this equivalence, in some cases, it may be necessary to first add another projection with additional projection attributes. The following result shows an equivalence that can be used for this purpose.

Proposition 2. Let $r = (A, I)$ be a mapping relation and $P \subseteq A$ be a non-empty subset of the attributes of r . For every $P' \subseteq A$, it holds that

$$\text{Project}^P(r) = \text{Project}^P(\text{Project}^{P \cup P'}(r)).$$

Proof. Let $(A_1, I_1) = \text{Project}^P(r)$ and $(A_2, I_2) = \text{Project}^P(\text{Project}^{P \cup P'}(r))$, where $P' \subseteq A$. To show that $(A_1, I_1) = (A_2, I_2)$, we first note that, by Definition 11, $A_1 = A_2 = P$. Hence, it remains to show that $I_1 = I_2$.

To show that $I_1 \subseteq I_2$ we consider an arbitrary mapping tuple $t \in I_1$ and show that $t \in I_2$. Given $t \in I_1$ with Definition 11, it holds that there exists mapping tuple $t' \in r$ such that t is the restriction of t' to P ; i.e., $t = t'[P]$ and $t(a) = t'(a)$ for all $a \in P$. Considering $P \cup P' \subseteq A$, let $t'' = t'[P \cup P']$, which, by Definition 11, also means that $\text{dom}(t'') = P \cup P'$ and $t'' \in \text{Project}^{P \cup P'}(r)$. Thus, there exists a mapping tuple $t''' \in \text{Project}^P(\text{Project}^{P \cup P'}(r))$ such that i) $\text{dom}(t''') = P$ and ii) $t'''(a) = t''(a)$ for all $a \in P$ according to Definition 11. Now that we have $\text{dom}(t''') = P = \text{dom}(t)$, and $t'''(a) = t''(a) = t'(a) = t(a)$ for all $a \in P$, we can conclude that $t''' = t$ and, thus, $t \in \text{Project}^P(\text{Project}^{P \cup P'}(r))$; i.e., $t \in I_2$.

Lastly, we show that $I_2 \subseteq I_1$ by considering an arbitrary mapping tuple $t \in I_2$ and prove that $t \in I_1$. Given $t \in I_2$ with Definition 11, it holds that i) $\text{dom}(t) = P$, and ii) there exists a mapping tuple $t' \in \text{Project}^{P \cup P'}(r)$ such that $t(a) = t'(a)$ for all $a \in P$. Since $t' \in \text{Project}^{P \cup P'}(r)$, it holds that i) $\text{dom}(t') = P \cup P'$ and ii) there exists a mapping tuple $t'' \in r$ such that $t'(a) = t''(a)$ for all $a \in P \cup P'$. Considering $P \subseteq A$, let $t''' = t''[P]$, which, by Definition 11, $\text{dom}(t''') = P$ and $t''' \in \text{Project}^P(r)$. Combining everything, we have i) $\text{dom}(t''') = P = \text{dom}(t)$ and ii) $t'''(a) = t''(a) = t'(a) = t(a)$ for all $a \in P$. Consequently, $t''' = t$ and $t \in \text{Project}^P(r)$; i.e., $t \in I_1$. $\square$

As a corollary of the previous two results, it is possible to push some projection attributes into an extend-based subexpression even in cases in which the condition for the equivalence in Proposition 1 is not directly satisfied.

Corollary 1. Let $r = (A, I)$ be a mapping relation, $a \in \mathcal{A}$ be an attribute such that $a \notin A$, φ be an extend expression, and $P \subseteq A \cup \{a\}$. It holds that

$$\text{Project}^P(\text{Extend}_\varphi^a(r)) = \text{Project}^P(\text{Extend}_\varphi^a(\text{Project}^{P'}(r))),$$

where $P' = (P \setminus \{a\}) \cup (\text{attrs}(\varphi) \cap A)$.

We emphasize that, while the equivalence in Corollary 1 does not eliminate the initial project operator, applying another projection already before the extend operator may still lead to efficiency gains due to reduced memory requirements, in particular if the added projection can be pushed down further.

While further equivalences for projection pushing can be shown (e.g., into a join), we also want to highlight the existence of cases in which a project operator can be removed completely. For instance, every project operator with a source operator as input may be removed, as shown by the following result.

Proposition 3. Let $s = (\text{type}, D)$ be a data source such that $\text{type} = (\mathcal{D}^{\text{as}}, \mathcal{D}^{\text{ci}}, \mathcal{D}^{\text{cz}}, L, L', \text{eval}, \text{eval}', \text{cast})$, $q \in L$, $\mathbb{P}$ be a partial function $\mathbb{P}: \mathcal{A} \rightarrow L'$, and $P \subseteq \text{dom}(\mathbb{P})$. It holds that

$$\text{Project}^P(\text{Source}^{(s,q,\mathbb{P})}) = \text{Source}^{(s,q,\mathbb{P}')}$$

where $\mathbb{P}'$ is the restriction of $\mathbb{P}$ to P ; i.e., $\text{dom}(\mathbb{P}') = P$ and $\mathbb{P}'(a) = \mathbb{P}(a)$ for all $a \in \text{dom}(\mathbb{P}')$.

Proof. Let $(A, I) = \text{Project}^P(\text{Source}^{(s,q,\mathbb{P})})$, and $(A', I') = \text{Source}^{(s,q,\mathbb{P}')}$, where $P = \text{dom}(\mathbb{P}')$. To show that $(A, I) = (A', I')$, we notice that, by Definition 11, $A = P$, and by Definition 6, $A' = \text{dom}(\mathbb{P}') = P$. Hence, we have $A = A'$ and, thus, it remains to show that $I = I'$.

We first show that $I \subseteq I'$ by considering an arbitrary mapping tuple $t \in I$ and show that $t \in I'$. Given $t \in I$ with Definition 11, it holds that there exists a mapping tuple $t'_d \in \text{Source}^{(s,q,\mathbb{P})}$ such that t is the restriction of t'_d to P ; i.e., $t = t'_d[P]$, $\text{dom}(t) = P$, and $t(a) = t'_d(a)$ for all $a \in P$. Since $t'_d \in \text{Source}^{(s,q,\mathbb{P})}$ with Definition 6, it holds that i) $d \in \text{eval}(D, q)$, ii) $X_d = \times_{a \in \text{dom}(\mathbb{P})} \{(a, v) \mid v \in \text{eval}'(D, d, q') \text{ with } q' = \mathbb{P}(a)\}$, iii) $\text{dom}(t'_d) = \text{dom}(\mathbb{P})$, and iv) $t'_d(a_i) = \text{cast}(v_i)$, with $((a_1, v_1), \dots, (a_n, v_n)) \in X_d$, $n = |\text{dom}(\mathbb{P})|$, and $\{a_1, \dots, a_n\} = \text{dom}(\mathbb{P})$. Considering $\mathbb{P}'$ is the restriction of $\mathbb{P}$ to P , there exists a mapping tuple $t''_d \in \text{Source}^{(s,q,\mathbb{P}')}$, with Definition 6, such that $\text{dom}(t''_d) = P$, and $t''_d(a) = t'_d(a)$ for all $a \in P$. Combining all the properties that we have shown, we can conclude that $\text{dom}(t) = \text{dom}(t''_d) = P$, and $t(a) = t'_d(a) = t''_d(a)$ for all $a \in P$ and, thus, $t \in \text{Source}^{(s,q,\mathbb{P}')} ; \text{ i.e., } t \in I'.$

Lastly, we show that $I' \subseteq I$ by considering an arbitrary mapping tuple $t_d \in I'$ and prove that $t_d \in I$. Given $t_d \in I'$ with Definition 6, and $\text{dom}(\mathbb{P}') = P$, it holds

that i) $d \in \text{eval}(D, q)$, ii) $X_d = \times_{a \in \text{dom}(\mathbb{P}')} \{(a, v) \mid v \in \text{eval}'(D, d, q') \text{ with } q' = \mathbb{P}'(a)\}$, iii) $\text{dom}(t_d) = \text{dom}(\mathbb{P}') = P$, and iv) $t_d(a_i) = \text{cast}(v_i)$, with $((a_1, v_1), \dots, (a_n, v_n)) \in X_d$. Now, provided with the conditions $\text{dom}(\mathbb{P}') = P$ and $P \subseteq \text{dom}(\mathbb{P})$, consider a mapping tuple t'_d for which it holds that i) $\text{dom}(t'_d) = \text{dom}(\mathbb{P})$, ii) $t'_d(a)$ is defined for all $a \in \text{dom}(\mathbb{P})$, and iii) $t'_d(a') = t_d(a')$ for all $a' \in \text{dom}(P)$. Thus, by Definition 6, $t'_d \in \text{Source}^{(s, q, \mathbb{P})}$. Next, let t'' be a mapping tuple that is the restriction of t'_d to P ; i.e, $t'' = t'_d[P]$, which, by Definition 11, means that $t'' \in \text{Project}^P(\text{Source}^{(s, q, \mathbb{P})})$. Combining everything together, we have that i) $\text{dom}(t'') = P = \text{dom}(t_d)$, and ii) $t''(a) = t'_d(a) = t_d(a)$ for all $a \in P$. As a consequence, $t'' = t_d$ and, thus, $t_d \in \text{Project}^P(\text{Source}^{(s, q, \mathbb{P})})$; i.e, $t_d \in I$. $\square$

As a final (and not particularly surprising) example, a project operator may also be removed completely (anywhere in a mapping plan) if its set of projection attributes coincides with the attributes of the input relation.

Proposition 4. For every mapping relation $r = (A, I)$, it holds that

$$\text{Project}^A(r) = r.$$

6.2 Pushing or Pulling Extend Operators

For the second group of equivalences that may be useful when optimizing mapping plans, we focus on extend operators that may be placed either on top of a join-based subplan or under the corresponding join operator (i.e., into one of the two inputs of the join). This decision may affect the efficiency of the plan. For instance, if the mapping relation r_1 in $\text{EqJoin}^{\mathbb{J}}(\text{Extend}_{\varphi}^a(r_1), r_2)$ contains many tuples but only very few of them have join partners in r_2 and, thus, the result of this join consists of a few tuples only, then it would be more efficient to extend only these few tuples instead of extending all tuples of r_1 , including all the ones that are essentially filtered out by the join. In other words, it would be more efficient to pull the extend operator out of the join. In contrast, however, if most of the tuples in r_1 have multiple join partners in r_2 and, thus, the result of the join contains many more tuples than r_1 , then it would be more efficient to apply the extend operator directly to r_1 rather than pulling it out of the join. Hence, depending on the selectivity of the join, a plan optimizer may decide to keep an extend operator under a join, to pull it out, or also to push an extend optimizer that is currently on top of a join into that join. The following result shows an equivalence that can be used for this purpose (in either direction).

Proposition 5. Let $r_1 = (A_1, I_1)$ and $r_2 = (A_2, I_2)$ be mapping relations such that $A_1 \cap A_2 = \emptyset$, let $\mathbb{J} \subseteq A_1 \times A_2$, let $a \in \mathcal{A}$ be an attribute such that $a \notin A_1 \cup A_2$, and let φ be an extend expression. If $\text{attrs}(\varphi) \cap A_2 = \emptyset$, then it holds that

$$\text{Extend}_{\varphi}^a(\text{EqJoin}^{\mathbb{J}}(r_1, r_2)) = \text{EqJoin}^{\mathbb{J}}(\text{Extend}_{\varphi}^a(r_1), r_2),$$

and if $\text{attrs}(\varphi) \cap A_1 = \emptyset$, then it holds that

$$\text{Extend}_{\varphi}^a(\text{EqJoin}^{\mathbb{J}}(r_1, r_2)) = \text{EqJoin}^{\mathbb{J}}(r_1, \text{Extend}_{\varphi}^a(r_2)).$$

The option to push a particular extend operator into a join (as per Proposition 5) requires this operator to be directly on top of the join in the mapping plan. Similarly, to pull an extend operator out of a join, this operator must be directly under the join. As this may not always be the case, a plan optimizer may try to move a relevant extend operator closer to a join (from either direction). The equivalence of Proposition 1 may be used for this purpose. Another option is to swap two subsequent extend operators, as shown by the following result.

Proposition 6. Let $r = (A, I)$ be a mapping relation, $a_1, a_2 \in \mathcal{A}$ be two different attributes that are not in A (i.e., $a_1 \neq a_2$ and $a_1, a_2 \notin A$), and φ_1 and φ_2 be two extend expressions. If $a_1 \notin \text{attrs}(\varphi_2)$ and $a_2 \notin \text{attrs}(\varphi_1)$, it holds that

$$\text{Extend}_{\varphi_1}^{a_1}(\text{Extend}_{\varphi_2}^{a_2}(r)) = \text{Extend}_{\varphi_2}^{a_2}(\text{Extend}_{\varphi_1}^{a_1}(r)).$$

7 Concluding Remarks and Future Work

In this paper, we provide a set of language-agnostic algebra for formalizing the semantics of mapping heterogeneous types of data sources to KG. Additionally, we show that our formalism is at least as expressive as RML for mapping heterogeneous data to KG, and as a consequence, we formalize the semantics of RML. Lastly, we provide a set of algebraic rewriting rules to show that mapping plans generated, as a result of our formalism, are optimizable. As future work, this algebraic foundation opens the door to three frontiers in the domain of KG generation from heterogeneous types of data sources; i) provably correct implementations of mapping engines, ii) further studies into the optimizations of mapping plans based on this formalism, and iii) expansion upon this formalism to capture the semantics of all existing mapping languages.

Acknowledgments. A bold run-in heading in small font size at the end of the paper is used for general acknowledgments, for example: This study was funded by X (grant number Y).

Disclosure of Interests. It is now necessary to declare any competing interests or to specifically state that the authors have no competing interests. Please place the statement with a bold run-in heading in small font size beneath the (optional) acknowledgments³, for example: The authors have no competing interests to declare that are relevant to the content of this article. Or: Author A has received research grants from Company W. Author B has received a speaker honorarium from Company X and owns stock in Company Y. Author C is a member of committee Z.

References

1. Arenas-Guerrero, J., Chaves-Fraga, D., Toledo, J., Pérez, M.S., Corcho, O.: Morph-KGC: Scalable knowledge graph materialization with mapping partitions. Semantic Web pp. 1–20 (Aug 2022). <https://doi.org/10.3233/sw-223135>

³ If EquinOCS, our proceedings submission system, is used, then the disclaimer can be provided directly in the system.

2. Asprino, L., Daga, E., Gangemi, A., Mulholland, P.: Knowledge graph construction with a façade: a unified method to access heterogeneous data sources on the web. *ACM Transactions on Internet Technology* **23**(1), 1–31 (2023)
3. Bischof, S., Decker, S., Krennwallner, T., Lopes, N., Polleres, A.: Mapping between RDF and XML with XSPARQL. *Journal on Data Semantics* **1**(3), 147–185 (2012)
4. Daga, E., Asprino, L., Mulholland, P., Gangemi, A.: Facade-X: An Opinionated Approach to SPARQL Anything. In: *Further with Knowledge Graphs – Proceedings of the 17th International Conference on Semantic Systems*, 6–9 September 2021, Amsterdam, The Netherlands. *Studies on the Semantic Web*, vol. 53, pp. 58–73. IOS Press (2021). <https://doi.org/10.3233/SSW210035>
5. Dimou, A., Vander Sande, M., Colpaert, P., Verborgh, R., Mannens, E., Van de Walle, R.: RML: A Generic Language for Integrated RDF Mappings of Heterogeneous Data. In: *Proceedings of the 7th Workshop on Linked Data on the Web*. *CEUR Workshop Proceedings*, vol. 1184. CEUR (Apr 2014), http://ceur-ws.org/Vol-1184/ldow2014_paper_01.pdf
6. Dimou, A., Vander Sande, M., De Meester, B., Heyvaert, P., Delva, T.: RDF Mapping Language (RML). Version 1.1.2; online at <https://rml.io/specs/rml/v1.1.2/> (2024)
7. Freund, M., Schmid, S., Dorsch, R., Harth, A.: FlexRML: A Flexible and Memory Efficient Knowledge Graph Materializer. In: *The Semantic Web*. pp. 40–56. Springer Nature Switzerland, Cham (2024)
8. Iglesias, E., Jozashoori, S., Vidal, M.E.: Scaling up knowledge graph creation to large and heterogeneous data sources. *Journal of Web Semantics* **75** (Jan 2023). <https://doi.org/10.1016/j.websem.2022.100755>
9. Iglesias-Molina, A., Assche, D.V., Arenas-Guerrero, J., Meester, B.D., Debruyne, C., Jozashoori, S., Maria, P., Michel, F., Chaves-Fraga, D., Dimou, A.: The RML Ontology: A Community-Driven Modular Redesign After a Decade of Experience in Mapping Heterogeneous Data to RDF. In: *Proceedings of the 22nd International Semantic Web Conference (ISWC)*. *Lecture Notes in Computer Science*, vol. 14266, pp. 152–175. Springer (2023). https://doi.org/10.1007/978-3-031-47243-5_9, https://doi.org/10.1007/978-3-031-47243-5_9
10. Lefrançois, M., Zimmermann, A., Bakeraally, N.: A SPARQL Extension for Generating RDF from Heterogeneous Formats. In: *The Semantic Web 14th International Conference, ESWC 2017, Portorož, Slovenia, May 28 – June 1, 2017, Proceedings*. pp. 35–50. Springer International Publishing, Portoroz, Slovenia (May 2017). https://doi.org/10.1007/978-3-319-58068-5_3
11. Lopes, N., Bischof, S., Decker, S., Polleres, A.: On the semantics of heterogeneous querying of relational, XML and RDF data with XSPARQL. In: *Proceedings of the 15th Portuguese Conference on Artificial Intelligence (EPIA 2011)*, Lisbon, Portugal. pp. 10–13. Citeseer (2011)
12. Min Oo, Sitt and De Meester, Ben and Taelman, Ruben and Colpaert, Pieter: Towards algebraic mapping operators for knowledge graph construction. p. 5 (2023)
13. Rodríguez-Muro, M., Rezk, M.: Efficient SPARQL-to-SQL with R2RML mappings. *Journal of Web Semantics* **33**, 141–169 (2015). <https://doi.org/https://doi.org/10.1016/j.websem.2015.03.001>, ontology-based Data Access
14. Rojas, J.A., Aguado, M., Vasilopoulou, P., Velitchkov, I., Van Assche, D., Colpaert, P., Verborgh, R.: Leveraging semantic technologies for digital interoperability in the European railway domain. In: *International Semantic Web Conference*. pp. 648–664. Springer (2021)

15. Sakor, A., Jozashoori, S., Niazmand, E., Rivas, A., Bougiatiotis, K., Aisopos, F., Iglesias, E., Rohde, P.D., Padiya, T., Krithara, A., Paliouras, G., Vidal, M.E.: Knowledge4COVID-19: A semantic-based approach for constructing a COVID-19 related knowledge graph from various sources and analyzing treatments' toxicities. *Journal of Web Semantics* **75**, 100760 (2023). <https://doi.org/https://doi.org/10.1016/j.websem.2022.100760>
16. Scrocca, M., Comerio, M., Carenini, A., Celino, I.: Turning Transport Data to Comply with EU Standards While Enabling a Multimodal Transport Knowledge Graph. *The Semantic Web – ISWC 2020* pp. 411–429 (2020). https://doi.org/10.1007/978-3-030-62466-8_26
17. Sequeda, J.F.: On the semantics of R2RML and its relationship with the direct mapping. In: *Proceedings of the 12th International Semantic Web Conference (Posters & Demonstrations Track) - Volume 1035*. p. 193–196. ISWC-PD '13, CEUR-WS.org, Aachen, DEU (2013)
18. Sitt Min Oo, Haesendonck, G., De Meester, B., Dimou, A.: RMLStreamer-SISO: An RDF Stream Generator from Streaming Heterogeneous Data. In: *The Semantic Web – ISWC 2022*. pp. 697–713. Springer, Springer International Publishing, Cham (2022). https://doi.org/10.1007/978-3-031-19433-7_40
19. Sundara, S., Das, S., Cyganiak, R.: R2RML: RDB to RDF Mapping Language. W3C recommendation, W3C (Sep 2012), <https://www.w3.org/TR/2012/REC-r2rml-20120927/>
20. Tailhardat, L., Chabot, Y., Troncy, R.: NORIA-O: An Ontology for Anomaly Detection and Incident Management in ICT Systems. In: *The Semantic Web*. pp. 21–39. Springer Nature Switzerland, Cham (2024)
21. Van Assche, D., Delva, T., Haesendonck, G., Heyvaert, P., De Meester, B., Dimou, A.: Declarative RDF graph generation from heterogeneous (semi-)structured data: A systematic literature review. *Journal of Web Semantics* (2022). <https://doi.org/10.1016/j.websem.2022.100753>
22. Xu, Z., Cruz, M.J., Guevara, M., Wang, T., Deshpande, M., Wang, X., Li, Z.: Retrieval-Augmented Generation with Knowledge Graphs for Customer Service Question Answering. In: *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. p. 2905–2909. SIGIR '24, Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3626772.3661370>